

Algoritmos de Classificação

3º Ano do Ensino Médio | Data Science & Inteligência Artificial | Material de Aula

Objetivo: Compreender a intuição, o funcionamento e o uso prático dos principais algoritmos de classificação, aplicando-os com scikit-learn em problemas com múltiplas features.

Algoritmo	Complexidade	Normalizar?	Fronteira	Probabilidade?
Regressão Logística	Baixa	Recomendado	Linear	Sim
KNN Classificador	Baixa	Obrigatório	Não-linear	Sim (parcial)
Árvore de Decisão	Média	Não necessário	Não-linear	Sim (folhas)
Random Forest	Alta	Não necessário	Não-linear	Sim (média das árvores)
Naive Bayes	Baixa	Não necessário	Linear/Gaussiana	Sim

1. O que é Classificação?

Classificação é um tipo de aprendizado supervisionado em que o objetivo é prever a qual categoria um exemplo pertence. Diferente da regressão (que prevê um número contínuo), a classificação prevê rótulos: spam ou não-spam, doente ou saudável, gato ou cachorro.

Exemplos do cotidiano

- E-mail é spam? → Sim / Não
- Foto contém gato? → Gato / Cachorro
- Tumor é maligno? → Maligno / Benigno
- Dígito manuscrito → 0, 1, 2 ... 9

Classificação vs. Regressão

- Regressão: saída é número contínuo → "Quanto custa? R\$ 320.000"
- Classificação binária: duas classes → Sim / Não
- Classificação multiclasse: N classes → Cachorro / Gato / Pássaro

2. Regressão Logística

Apesar do nome, é um algoritmo de classificação! Calcula a probabilidade de um exemplo pertencer a uma classe. Se a probabilidade for maior que 50%, classifica como positivo; caso contrário, como negativo.

$$P(\text{classe}=1) = \text{sigmoid}(z)$$

$$z = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n$$

sigmoid transforma qualquer número em valor entre 0 e 1

Intuição

Pense num termômetro de probabilidade: quanto mais evidências de spam (palavras suspeitas, remetente desconhecido), mais o

ponteiro se aproxima de 100%. A regressão logística aprende quais evidências pesam mais.

Código com scikit-learn

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# X tem >= 2 colunas (features)
X = df[["horas_estudo", "faltas", "media_anterior"]]
y = df["aprovado"] # 0 = reprovado, 1 = aprovado

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

modelo = LogisticRegression()
modelo.fit(X_train, y_train)

# Ver a probabilidade de cada exemplo
probas = modelo.predict_proba(X_test)
print("Prob. reprovado / aprovado:", probas[:3])

y_pred = modelo.predict(X_test)
```

Quando usar

- Fronteira entre classes aproximadamente linear
- Precisa de probabilidades (não só a classe)
- Baseline rápido e interpretável
- Dados com features bem definidas

Cuidados

- Não captura relações muito não-lineares
- Sensível a outliers extremos
- Sempre normalizar os dados

3. KNN Classificador

Classifica um exemplo com base nos K vizinhos mais próximos no espaço de features. A classe mais comum entre esses vizinhos vira a previsão. Não há fase de treinamento formal — é busca pura nos dados (lazy learner).

Intuição: Para saber se um e-mail é spam, encontre os K e-mails mais parecidos no histórico e veja a classe da maioria. Se 6 dos 7 vizinhos são spam, o e-mail provavelmente também é. Simples — mas os dados precisam estar normalizados, senão features com valores maiores dominam o cálculo de distância.

ATENÇÃO: Normalização é obrigatória no KNN! Sem normalizar, uma feature como "renda" (0–10000) vai dominar completamente a distância sobre "número de filhos" (0–5). O StandardScaler coloca todas as features na mesma escala antes do cálculo.

Código com scikit-learn

```
from sklearn.neighbors import KNeighborsClassifier

# KNN REQUER normalização — use os dados já escalados
modelo = KNeighborsClassifier(
    n_neighbors=7,          # número de vizinhos
    weights="distance"     # vizinhos mais próximos pesam mais
)
modelo.fit(X_train_sc, y_train)  # usa dados normalizados

y_pred = modelo.predict(X_test_sc)

# Ver probabilidade por classe para um exemplo
probas = modelo.predict_proba(X_test_sc[0:1])
print("Probabilidades por classe:", probas)

# Ver os K vizinhos de um ponto específico
dists, indices = modelo.kneighbors(X_test_sc[0:1])
print("Distâncias:", dists)
print("Índices vizinhos:", indices)
```

Parâmetro	O que controla	Sugestão inicial
n_neighbors	Quantos vizinhos votar na classificação	5 a 11 (ímpar)
weights	"uniform": todos iguais / "distance": pesam pela proximidade	"distance"
metric	Como medir distância entre exemplos	"euclidean"

4. Árvore de Decisão

Divide o espaço de features em regiões por meio de perguntas e classifica cada região com a classe majoritária nela. Funciona como um fluxograma de diagnóstico que o algoritmo constrói sozinho a partir dos dados.

Intuição: Imagine um médico fazendo perguntas: "A febre é acima de 38°C? Se sim, há tosse? Se sim, classifique como possível gripe." A árvore aprende quais perguntas fazer e em qual ordem para separar melhor as classes — minimizando a "bagunça" (impureza) em cada folha.

Código com scikit-learn

```
from sklearn.tree import DecisionTreeClassifier, export_text
```

```
# Árvore NÃO precisa de normalização
modelo = DecisionTreeClassifier(
    max_depth=4,          # profundidade máxima
    min_samples_leaf=10   # mín. de amostras por folha
)
modelo.fit(X_train, y_train) # sem scaler!

# Visualizar as regras aprendidas (muito didático!)
regras = export_text(modelo, feature_names=list(X.columns))
print(regras)

# Importância de cada feature
for feat, imp in zip(X.columns, modelo.feature_importances_):
    print(f"{feat}: {imp:.3f}")

y_pred = modelo.predict(X_test)

# Probabilidade de cada classe
probas = modelo.predict_proba(X_test)
```

Atenção: Underfitting × Overfitting

Árvore muito rasa (max_depth=1): underfitting — não consegue capturar o padrão.

Árvore muito profunda (sem limite): overfitting — decora os dados de treino e erra nos novos. O objetivo é encontrar a profundidade certa.

Parâmetro	O que controla	Sugestão inicial
max_depth	Profundidade máxima da árvore	3 a 6
min_samples_leaf	Mínimo de amostras numa folha	5 a 20
min_samples_split	Mínimo para dividir um nó	10 a 30

5. Naive Bayes

Usa o Teorema de Bayes para calcular a probabilidade de cada classe dado o conjunto de features. Assume que as features são independentes entre si — daí o "naive" (ingênuo). Na prática, mesmo quando essa suposição não é totalmente verdadeira, o modelo funciona surpreendentemente bem.

Intuição

Pense num detector de spam: o modelo aprende que palavras como "promoção", "grátis" e "clique aqui" aparecem muito mais em spam do que em e-mails normais. Para classificar um novo e-mail, ele "multiplica as evidências" de cada palavra e escolhe a classe com maior probabilidade resultante.

A fórmula central

$$P(\text{classe} \mid \text{features}) \propto P(\text{classe}) \times P(f_1 \mid \text{classe}) \times P(f_2 \mid \text{classe}) \times \dots$$

Cada feature contribui de forma independente para a probabilidade final da classe.

Código com scikit-learn

```
# Escolha a variante conforme o tipo de dado:
# GaussianNB → features numéricas contínuas
# MultinomialNB → contagens (ex: frequência de palavras)
# BernoulliNB → features binárias (0/1)

from sklearn.naive_bayes import GaussianNB

# Naive Bayes NÃO precisa de normalização
modelo = GaussianNB()
modelo.fit(X_train, y_train) # sem scaler!

y_pred = modelo.predict(X_test)

# Probabilidades por classe (ponto forte do NB!)
probas = modelo.predict_proba(X_test)
print("Probabilidades:", probas[:3])

# Para classificação de texto (bag-of-words):
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer
vec = CountVectorizer()
X_vec = vec.fit_transform(textos)
modelo_texto = MultinomialNB()
```

Quando usar

- Datasets grandes — treino muito rápido
- Classificação de texto (spam, sentimentos)
- Quando probabilidades confiáveis são importantes
- Como baseline rápido

Cuidados

- A suposição de independência raramente é 100% real
- Pode ser superado por modelos mais sofisticados
- Sensível a features muito correlacionadas

6. Métricas de Avaliação

A acurácia sozinha pode enganar! Se 95% dos e-mails não são spam, um modelo que chuta "não spam" para tudo tem 95% de acurácia — mas é inútil. Por isso usamos outras métricas.

Acurácia = acertos / total Proporção total de previsões corretas — funciona bem quando as classes são balanceadas.	Precisão = VP / (VP + FP) Dos exemplos classificados como positivos, quantos realmente são? Importante quando falsos alarmes têm custo alto.
Recall = VP / (VP + FN) Dos exemplos que realmente são positivos, quantos o modelo encontrou? Crucial quando não pode perder casos positivos (ex: doenças).	F1-Score = 2 × (P × R) / (P + R) Média harmônica entre Precisão e Recall — bom quando há desbalanceamento entre as classes.

Métrica	Melhor quando...	Cuidado!
---------	------------------	----------

Acurácia	Classes balanceadas	Enganosa em datasets desbalanceados
Precisão	Falso positivo é caro (ex: spam legítimo bloqueado)	Pode ignorar muitos casos positivos reais
Recall	Falso negativo é caro (ex: doença não detectada)	Pode gerar muitos alarmes falsos
F1-Score	Classes desbalanceadas, equilíbrio entre P e R	Não separa Precisão de Recall individualmente

Código com scikit-learn

```
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score,
                             classification_report, confusion_matrix)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average="weighted")
rec = recall_score(y_test, y_pred, average="weighted")
f1 = f1_score(y_test, y_pred, average="weighted")

print(f"Acurácia = {acc:.4f}")
print(f"Precisão = {prec:.4f}")
print(f"Recall = {rec:.4f}")
print(f"F1-Score = {f1:.4f}")

# Relatório completo por classe (muito útil!)
print(classification_report(y_test, y_pred))

# Matriz de confusão
print(confusion_matrix(y_test, y_pred))
# Dica: RMSE >> MAE indica a presença de outliers
```

7. Random Forest Classificador

Um conjunto (ensemble) de árvores de decisão. Cada árvore é treinada com uma amostra diferente dos dados e considera features diferentes. A classificação final é feita por votação majoritária entre todas as árvores.

Intuição — "Sabedoria das multidões": Imagine pedir a opinião de 100 especialistas. Cada um tem um ponto de vista ligeiramente diferente. Nenhum é perfeito, mas a classe que a maioria votou tende a ser muito mais confiável do que qualquer um individualmente. Esse é o princípio de bagging (bootstrap aggregating).

Código com scikit-learn

```
from sklearn.ensemble import RandomForestClassifier

# Random Forest não precisa de normalização
modelo = RandomForestClassifier(
    n_estimators=100, # número de árvores
```

```

    max_depth=6,          # profundidade por árvore
    random_state=42       # reprodutibilidade
)
modelo.fit(X_train, y_train)

# Importância das features (muito útil!)
import pandas as pd
importancias = pd.Series(
    modelo.feature_importances_,
    index=X.columns
).sort_values(ascending=False)
print(importancias)

y_pred = modelo.predict(X_test)

```

Aspecto	Árvore Única	Random Forest
Estabilidade	Baixa — sensível aos dados	Alta — votação suaviza instabilidade
Overfitting	Frequente sem limitação	Menos comum pela diversidade
Interpretabilidade	Sim (export_text)	Difícil (caixa-preta)
Velocidade de treino	Rápida	Mais lenta (N árvores)
Precisão típica	Mediana	Alta

8. Pipeline Completo (boas práticas)

Na prática, sempre use Pipeline para encadear pré-processamento e modelo. Isso evita o vazamento de dados (data leakage) entre treino e teste.

```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y # stratify mantém
    proporção das classes
)

# Pipeline: scaler + modelo em um só objeto
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("modelo", RandomForestClassifier(n_estimators=100, random_state=42))
])

# Validação cruzada (5 folds) — avaliação mais confiável
scores = cross_val_score(pipe, X, y, cv=5, scoring="f1_weighted")
print(f"F1 por fold: {scores.round(3)}")
print(f"F1 médio: {scores.mean():.3f} +/- {scores.std():.3f}")

# Treinar e avaliar no teste
pipe.fit(X_train, y_train)

```

```
y_pred = pipe.predict(X_test)
print(classification_report(y_test, y_pred))

# Salvar o modelo treinado
import joblib
joblib.dump(pipe, "meu_classificador.pkl")
```

9. Comparativo Geral — Guia de Escolha

Algoritmo	Dados pequenos	Não-linear	Interpretável	Sem normalizar	Velocidade
Reg. Logística	Ótimo	Não	Ótimo	Não	Muito rápida
KNN	Bom	Sim	Moderado	Não*	Lenta na predição
Árvore de Decisão	Bom	Sim	Bom	Sim	Rápida
Naive Bayes	Ótimo	Parcial	Moderado	Sim	Muito rápida
Random Forest	Moderado	Ótimo	Baixo	Sim	Moderada

*** KNN exige normalização obrigatoriamente.** Sem normalizar, features com valores numericamente maiores dominam o cálculo da distância e o modelo se torna enviesado.

10. Exercícios Propostos

Exercício 1 — Regressão Logística

Use o dataset Iris (`sklearn.datasets.load_iris`) para classificar espécies de flores. Calcule acurácia, precisão, recall e F1-Score. Quais features têm maior coeficiente?

Exercício 2 — Comparação

Com o mesmo dataset, treine `LogisticRegression`, `DecisionTreeClassifier` e `RandomForestClassifier`. Compare o F1-Score de cada um. Qual foi o melhor? Por quê?

Exercício 3 — Overfitting na Árvore

Com uma `DecisionTreeClassifier`, varie `max_depth` de 1 a 15. Plote a acurácia de treino e de teste para cada profundidade. Onde ocorre o overfitting?

Exercício 4 — Pipeline com KNN

Construa um Pipeline com `StandardScaler` + `KNeighborsClassifier`. Use `cross_val_score` com 5 folds. Varie `n_neighbors` de 1 a 20 e encontre o melhor K.

Desafio — Naive Bayes para Texto

Baixe um dataset de avaliações de filmes (positivas/negativas). Use CountVectorizer + MultinomialNB para criar um classificador de sentimentos. Compare com os outros algoritmos: qual performa melhor no texto?